Service HLD

Intro

High-Level Design (HLD) for the disease risk identification.

Overview

Clinical decisions and research often require fast identification of disease risk indicators from biomedical literature. Example: thrombosis risk factors include obesity, COVID-19 infection, prolonged immobility, certain genetic mutations, and medications. These indicators are scattered across papers and often phrased variably. We need a system that:

- Ingests papers at scale, normalizes key entities and relations, and keeps provenance.
- Supports fast, accurate retrieval of risk indicators per disease, population, treatment, and context.
- Surfaces answers with citations and confidence, not free text alone.

Primary objectives:

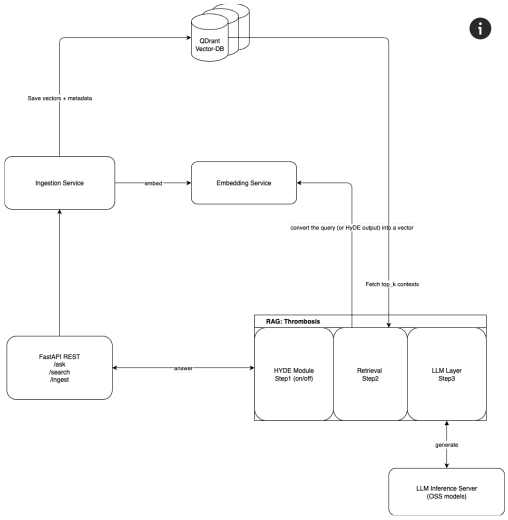
- Retrieval for queries like: "List established risk factors for venous thrombosis in adults, with evidence strength and citations."
- High recall with controlled hallucination risk through grounded retrieval.
- Extensibility to all illnesses and future data sources.
    - Food for though: fine tuning more LLMs per super set of disease.
    - Agents: One coordinator agent can handle those LLMs disease.

Scope

In scope:

- Literature ingestion from PubMed or others full text where legally allowed for Thrombosis risk factors (Ingestion functionality).
- Embedding Service that converts text into numeric vectors.
- RAG answer generation with citations, confidence, and structured JSON including the Risk Factors.
- APIs: REST (FastAPI)

HLD



Components - Services

API layer

The system's single entry point the front door.

What it does:

- Basic example of the API functionality:
  - `/ask` - Generate an answer grounded in the stored papers, it uses the LLM to prepare the final answer.
  - `/search` - Semantic search. No generation from LLM.
  - `/ingest/upload` - Upload one PDF paper.
  - `/ingest/items` - Accept one or many textual documents from JSON.
    - 202 - Accept a batch of structured text documents for ingestion, enqueue them for background processing, and immediately return a job reference so clients can track progress asynchronously.

    - `/ingest/jobs/{id}` - Returns overall and per-item state.
    - `/ingest/{fingerprint}` - Existence check.
- Sends the request to the correct internal part (RAG or Ingestion).
- Returns the final answer or search results to the user.

This is how users talk to the system. Think of it like the receptionist that routes requests to the right department.

Embedding Service

An internal service that converts text into numeric vectors so the system can compare meanings quickly.

What it does:

- Receives text from two places:
    - RAG pipeline: the user query or the HyDE draft.
    - Ingestion Service: the text chunks extracted from PDFs.
- Calls a chosen embedding model (OpenAI or a Hugging Face model).
- Returns vectors plus basic metadata (model id, dimension, timing).
- Hides model details from the rest of the system so you can swap models later without changes elsewhere.

Inputs:

- List of short texts from RAG or Ingestion.

Outputs:

- One vector per text.

Simple contract:

- Request: texts[], model
- Response: vectors[][], dim, model
    - dim: short for dimension refers to the length of each embedding vector produced by the model

Ingestion Service

The controlled way to add new knowledge. It only accepts uploads that arrive through FastAPI and only PDFs of scientific papers.

What it does:

1. Accepts a PDF file and minimal metadata from FastAPI.
2. Extracts text from the PDF and splits it into smaller pieces that are easy to search.
3. Sends those pieces to the Embedding Service to get vectors.
4. Stores vectors and metadata in Qdrant so they become searchable.
5. Reports back how many chunks were added and if any duplicates were skipped.

Inputs:

- A PDF file uploaded by a user plus metadata like title, year, disease, tags.

Outputs:

- Status report: inserted chunks, skipped duplicates, any errors.

Key rules:

- No external crawling. Only user uploads via FastAPI are accepted.
- Deduplicate by file fingerprint so the same paper is not ingested twice.
- Metadata should include: source=pubmed, pmid if available, title, authors, year, disease, tags.

What success looks like:

- Upload returns quickly while background processing finishes.
- Paper appears in search results within minutes.
- If the same PDF is uploaded again, it is recognized and skipped.

Typical steps for one PDF:

- Receive file from FastAPI.
- Compute a fingerprint.
- Extract text.
- Split into chunks.
- Call Embedding Service.
- Upsert points into Qdrant with metadata.

RAG pipeline

RAG: Thrombosis (or any domain RAG)

The part that loads new medical documents into the system.

The components in the RAG are:

- **HyDE Module**: writes a short hypothetical answer to help find better results. HyDE is part of the LangChain framework.

    - A helper that improves search quality, it takes the user question and writes a “fake” answer first. This helps the system find more relevant context in Qdrant. HyDE generates synthetic text only. **No DB access.**
    - Before looking for answers, it imagines what the answer might look like making search smarter.
- **Retrieval Step**: searches Qdrant to get the most relevant text pieces is the search engine inside the RAG pipeline. Basically it finds the most useful information from the knowledge base for the AI to read before it answers.
    - Sends the query vector to Qdrant.
    - Collects the top matching document pieces.
    - Returns them to the LLM Layer to build the response.
- **LLM Layer**: uses those pieces to write the final answer.
    - Reads the user’s question and the top document results.
    - Builds a clear, complete answer.
    - Sends the answer to the FastAPI so the user sees it.
    - **LLM Inference Server**: Integration with the LLM Layer, this is an LLM like Llama, Gemini etc.
        - Where the actual large language model runs (the “AI brain”).
        - Receives prompts from the LLM Layer.
        - Generates text the final answer, summary, or explanation.
        - Returns it to the pipeline.
        - This is the powerful AI engine that creates the human-like answer, using all the supporting data found earlier.

Qdrant Vector DB

The searchable memory of the system. It holds the vectors and the document snippets that those vectors represent.

What it does:

- Stores one vector per text chunk with rich metadata.
- Finds the most relevant chunks for a question by similarity search.
- Filters results by metadata such as disease or publication year.

Inputs:

- Upserts from Ingestion Service: vectors + chunk text + metadata.

- Search requests from the RAG pipeline: a query vector, top_k, and filters.

Outputs:

- For search: a ranked list of chunks with text, metadata, and a relevance score.
- For ingestion: confirmation that points were inserted.

Data stored per chunk:

- vector
- text
- sha256 of the original PDF
  - DOI can be used for that instead of a hash or hash DOI.
- source=pubmed
- pmid
- title
- authors
- year
- disease
- tags
- chunk_ix
- DOI URL

Key rules:

- Use one collection for chunks.
- Build an index that makes nearest neighbor search fast.
- Keep metadata consistent so filtering works the same across all uploads.
- Do not delete existing vectors when you change the embedding model. Plan a controlled re-embed if needed.

How others interact with it:

- Ingestion Service writes new points.
- RAG Retrieval searches for top_k similar chunks with optional filters.
- RAG receives the top chunks and passes them to the LLM Layer.